

1 Undecidability

1.1 The Church-Turing thesis 丘奇图灵定理

丘奇图灵理论中，算法是一个在所有输入上都可以停机并且能够判定语言并且可计算的图灵机。不能被图灵机执行的计算任务是无法实现并且 undecidable。

这一章研究通用图灵机，针对我们之前的串行图灵机，虽然也和这个有点相似，但是之前的串行图灵机主要研究固定任务的简单衔接，而本章我们希望通过先定义图灵机的统一编码规则（也就是用同一台图灵机的某个语言对某个问题进行描述并且放入此通用图灵机能够直接运行），来实现机器处理自身编码能力，而非通过问题思考如何解决此问题来设计图灵机。

1.2 Universal Turing Machines 通用图灵机设计

- 设计通用图灵机

1. 在字母表中，我们对状态和带符号编码为字符串形式，一个表示图灵机状态的字符串形式是以 q 为开头后跟 0 到多个 0 或者 1 组成的字符串（ q 后跟着一个二进制字符串）；纸带上字符用 a 后跟二进制字符串进行表示。
2. 我们定义一个五元组的图灵机， i 满足 2 的 i 次方大于等于状态数量的最小整数， j 是满足 2 的 j 次方大于等于字符表数量 +2 的最小整数。同时，定义空一直是 $a0^j$ ，右三角号为 $a0^{j-1}1$ ，左箭头为 $a0j - 210$ ，右箭头为 $a0j - 211$ 。起始状态为 $q0^i$
3. 我们将图灵机表示为 $M, [M]$ 表示着 M 的状态转移表，转移表会写成四元组的形式 (q, a, p, b) ， q 表示当前状态， a 表示当前指针头扫描到的字符， p 表示下一个状态编码， b 表示要执行的动作（包括要写的字符以及指针头移动的方向）。
4. 起始状态 $(s, \sqcup)$ 。停机状态 H 通过找到 M 图灵机中转移方程四元组中不作为第一个分量的状态来间接确定。如果 M decides a language, $H = \{y, n\}$ 。
5. 这样任何图灵机都能被表示。并且任何属于字符串子集的的字符串都有一个唯一的表示，denoted 为 w 。

6. eg1

state, symbol	δ
s a	$(q, \sqcup)$
s $\sqcup$	$(h, \sqcup)$
s $\triangleright$	$(s, \rightarrow)$
q a	(s, a)
q $\sqcup$	$(s, \rightarrow)$
q $\triangleright$	$(q, \rightarrow)$

state/symbol	representation
s	$q00$
q	$q01$
h	$q11$
$\sqcup$	$a000$
$\triangleright$	$a001$
$\leftarrow$	$a010$
$\rightarrow$	$a011$
a	$a100$

Thus, the representation of the string $\triangleright aa \sqcup a$ is

$$“\triangleright aa \sqcup a” = a001a100a100a000a100.$$

The representation “ M ” of the Turing machine M is the following string:

$$“M” = (q00, a100, q01, a000), (q00, a000, q11, a000), (q00, a001, q00, a011), \\ (q01, a100, q00, a011), (q01, a000, q00, a011), (q01, a001, q01, 011). \diamond$$

图 1: UniversalTM

- 如何使用通用图灵机，denoted U ，模拟其他图灵机

通用图灵机可以接受两个参数，一个是另一台图灵机 M 的描述“ M ”，另一个是输入字符串 w 的描述。如果我们要让 U 模拟 M 的行为，我们要满足： U 在 M, w 两个输入能够停机当且仅当 M 在输入 w 上停机。

$U([M], [w]_M) = [M(w)]_M$ 表明 w 是针对图灵机 M 的字母表和编码规则进行编码。在 w 的编码中，我们可以使用通用图灵机对 M 编码的 w 进行转换，并且能够计算。

1. 我们先使用三带通用图灵机 U' 模拟图灵机 M 的操作，直接使用单带图灵机对 M 模拟过于复杂，如果三带图灵机能够模拟出 M 的操作，由定理多带图灵机等价于单带图灵机，所以如果我们能够使用三带图灵机对 M 进行模拟，那么我们同样也能够使用单带图灵机对 M 进行模拟。

2. U' 的三条带子, 第一条纸带存储 M 当前的带内容, 第二条纸带存储 M 的描述让 U' 可以查阅 M 的状态方程, 第三条纸带存储 M 的当前状态编码。
3. 首先 U' 将 M 编码复制到 U' 的第二条纸带, 并且将输入 w 的编码移动到以 $\triangleright \sqcup$ 为起点的第一个纸带形成 $\triangleright \sqcup w$, U' 在第三条纸带上写上 M 的初始状态编码。
4. U' 模拟 M 的操作: 第一条纸带的读写头对准 M 当前扫描的符号所对应的编码字符串, 第二条纸带和第三条纸带在两次模拟之间会停留在纸带的左端。 U' 扫描第二条纸带, 直到找到第一个四元组, 四元组的第一个分量与第三条纸带上写入的分量相匹配, 第二个分量与第一条纸带上当前扫描到的符号相匹配, 若找到这样的四元组, U' 将状态更新为第三个分量, 并且在第一条纸带上执行第四个分量所指定的操作。若第四个分量是 M 带字母表中某一符号的编码, 则将符号编码串写入第一条纸带; 若第四条纸带是左移编码, U' 将第一条带子的读写头移至左侧第一个以 a 左侧第一个以 a 开头的符号编码处, 若遇到右三角号, 转换为空白符的编码。
5. 若某一步中, 状态符号组合不能满足第二条纸带中的任何四元组, 那么为停机状态。

1.3 The halting problem 停机问题

停机问题定义: 对于停机判定程序, 我们使用设计一个程序 (停机判定程序) 的语言设计程序 P 以及输入 X 。如果一个程序判断输入 X 是否会在输入的程序 P 停机, 那么我们称这个程序为 $\text{halts}(P, X)$ 。 $\text{Halt}_{TM} = \{ \langle M, w \rangle \mid \text{TM } M \text{ halts on input string } w \}$

- 如何证明不存在停机程序能够判断所有程序能够在某个输入停机?

证明 1

1. 设计一个对角程序 $\text{diagonal}(X)$: X 程序以自身为输入的停机程序, 如果此停机程序停机, 那么不断执行此程序, 如果此停机程序不停机, 则停机。如果 $\text{halt}(X, X) = \text{yes}$ 那么 $\text{diagonal}(X)$ 一直循环, 否则 $\text{diagonal}(X)$ 停机

2. 那么证明 1 中我们使用对角化的思想, 构造 $\text{diagonal}(\text{diagonal})$, 如果 $\text{diagonal}(\text{diagonal})$ 停机, $\text{halts}(\text{diagonal}, \text{diagonal})$ 不停机- $\rightarrow \text{diagonal}$ 在 diagonal 上不停机, 与我们假设 $\text{diagonal}(\text{diagonal})$ 矛盾。

证明 2

1. 定义停机问题 $H = \{ \langle M, w \rangle \mid \text{TM } M \text{ halts on input string } w \}$ 。我们知道 H 是递归可枚举的, 因为停机问题 H 能够被 universal turing machine 半判定, 就是如果输入属于我们的程序能够判定的, 那么就停机返回 yes, 如果不属于那么就不断循环, 我们可以使用通用模型 U 来模拟 H 的输入输出和状态。并且如果一个停机问题能够是 decidable, 那么所有程序在任意某个输入都会停机, 也就是说所有 recursive enumerable languages 都是可判定的。
 2. 我们假设 H 是可以被图灵机 M_0 判定, 给定一个图灵机 M 半判定语言 $L(M)$, 我们可以设计一个图灵机 M' 来判定 $L(M)$, 首先将 M' 输入改为 M_0 的输入格式 " M^w ", 并且模拟 M_0 的行为, 如果 M_0 accept w , 那么 M' accept w ; 如果 M_0 输出 no, 那么 M' 输出 no。
 3. 我们假设 $H_1 = \{ \langle M \rangle \mid \text{图灵机 } M \text{ 在输入为 } \langle M \rangle \text{ 上停机} \}$ 。
 4. 如果 H_1 是递归的, 那么由于闭包性它的补集也是递归的, $\overline{H_1} = \{ w \mid w \text{ 不是图灵机的编码或者 } w \text{ 是图灵机 } M \text{ 的编码 } \langle M \rangle \text{ 并且 } M \text{ 在输入 } \langle M \rangle \text{ 上不停机} \}$ 。如果 M^* 半判定 H_1 补集, 那么图灵机 M^* 的编码 " M^* " 并且 M^* 在输入 " M^* " 上不停机, M^* 不接受 " M^* "。因此得出矛盾, 停机问题不是可判定的, 但是是半判定的。
- 定理: 语言 H 不是递归的, 因此递归语言是可枚举递归语言类的真子集 (strict subset)
 - 递归可枚举语言类在补运算上是不封闭的。

1.4 Acceptance problem 接受问题

- Let $A_{TM} = \{ \langle M, w \rangle \mid M \text{ 是一个图灵机并且 } M \text{ 接受 } w \}$
 A_{TM} 是 undecidable 但是是图灵可识别的。不可判定只是说其没有判定器, 而不是说不可半判定。

证明：

1. assume that A_{TM} 是可判定问题以及 H 是 A_{TM} 的判定器。对于输入 (M, w) , M 是图灵机, w 是字符串。如果 M 接受 w , H 停机并且接受；如果 M 不接受 w , 那么 H 停机并且拒绝（分段函数写法）。
 2. 构建一个新的图灵机 D , $D = \text{"On input } \langle M \rangle, \text{ where } M \text{ is a TM"}:$
 1. Run H on input $\langle M, \langle M \rangle \rangle$.
 2. Output the opposite of what H outputs. If H accepts, D reject and if H reject, D accept.
 3. 输入描述 (D) 运行 D 图灵机, 如果 D 不接受 $\langle D \rangle$, 那么 $D(\langle D \rangle)$ 接受, 如果 D 接受 $\langle D \rangle$, 那么 $D(\langle D \rangle)$ 拒绝。
 4. 矛盾, 因此 $TM D$ 和 $TM H$ 不可以存在。
- A_{TM} 是 Turing-Recognizable 图灵可识别
- $U = \text{"On input } \langle M, w \rangle, \text{ where } M \text{ 是一个图灵机, } w \text{ 是一个字符串: 1. 输入为 } w, \text{ 模拟 } M \text{ 的结果 2. 如果 } M \text{ 接受 } w, \text{ 则接受, 如果 } M \text{ 拒绝, 则拒绝, 如果 } M \text{ 循环, 则循环}"$

1.5 Turing-Unrecognizable Language 图灵不可识别语言

- co-Turing-recognizable: 对于语言 L , L 的补给是图灵可识别的, 我们称 L 为 "co-Turing-recognizable"
- 一个语言是 decidable, 当且仅当此语言是图灵可识别以及 co-图灵可识别。

证明：

PROOF We have two directions to prove. First, if A is decidable, we can easily see that both A and its complement $\bar{A}$ are Turing-recognizable. Any decidable language is Turing-recognizable, and the complement of a decidable language also is decidable.

For the other direction, if both A and $\bar{A}$ are Turing-recognizable, we let M_1 be the recognizer for A and M_2 be the recognizer for $\bar{A}$. The following Turing machine M is a decider for A .

$M =$ “On input w :

1. Run both M_1 and M_2 on input w in parallel.
2. If M_1 accepts, *accept*; if M_2 accepts, *reject*.”

图 2: Turing-recognizable-decidable

- 定理: $\overline{A_{TM}}$ 不是图灵可识别的。根据定理, A_{TM} 是不可判定, 并且是图灵可识别的, 所以其补集为图灵不可识别的。